窓を広げる

MLP言語モデル — 記憶1文字の壁を越える

rkskmt

前回の振り返り

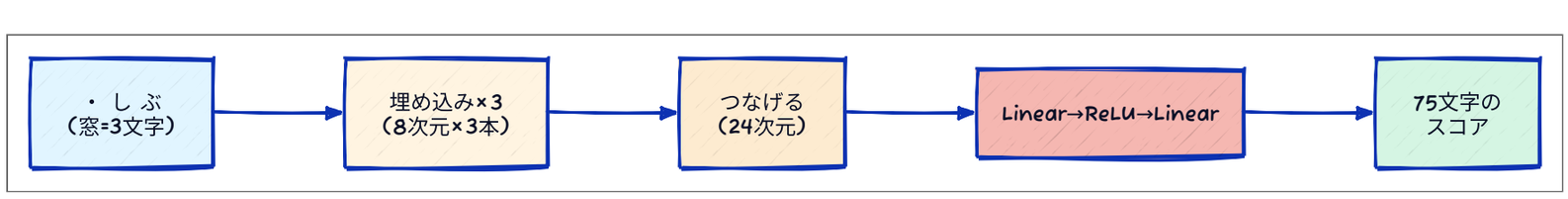
- 文字は**埋め込み**で d 次元ベクトルになった（表引き＝**`nn.Embedding`**）
- ただしモデルはまだバイグラム：**直前の1文字**しか見ていない
- 地名バイグラムの損失は**2.34の壁**に当たった — 作戦自体の限界だった

クイズ

「かみ」まで読んだ。次は？「かわ」まで読んだ。次は？ **2文字**見えるだけで、どれくらい予想が変わるだろう。

作戦：直前の数文字をまとめて入れる

- 直前 n 文字の**窓**（**context window**）を決めて、全部ベクトルにして、**つなげて**1本にする
- つないだベクトルを**多層ネットワーク**（MLP）に通す



- 部品は **`Embedding`**（前回の表引き）・**`Flatten`**（つなげるだけ）・そして **`Linear`** と **`ReLU`**

寄り道 — 多層ネットワーク再入門

- バイグラムのモデルは**線形1枚**（掛けて足すだけ）だった。今日はここに部品を足して「多層」にする
- なぜ足す必要があるのか — まず線形の弱点を自分の目で確かめる

クイズ

線形1枚で力不足なら、**線形を2枚重ねれば**賢くなるはず。本当か？

実験：線形×線形＝線形

Code

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
f = nn.Sequential(nn.Linear(2, 2, bias=False), nn.Linear(2, 2, bias=False))
W = f[1].weight @ f[0].weight        # 2枚の行列を先に掛けて1枚にまとめる

x = torch.randn(3, 2)
print(f(x))                          # 2枚に順に通した結果
print(x @ W.T)                       # まとめた1枚に通した結果
```

Result

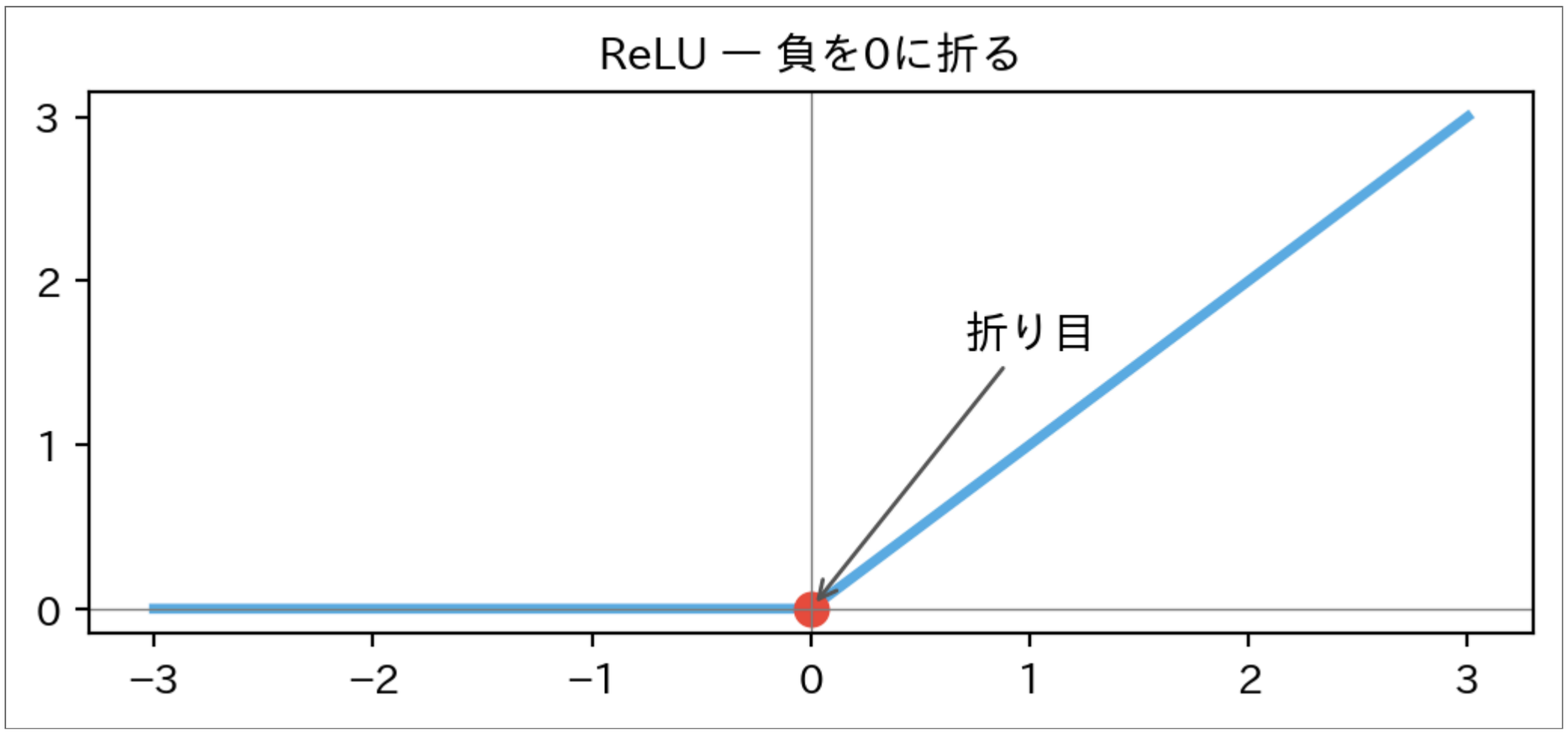
```
tensor([[ 0.4006,  0.7693],
        [-0.2132, -0.3805],
        [ 0.1598,  0.3545]], grad_fn=<MmBackward0>)
tensor([[ 0.4006,  0.7693],
        [-0.2132, -0.3805],
        [ 0.1598,  0.3545]], grad_fn=<MmBackward0>)
```

- 出力は**完全に一致** — **何枚重ねても、まとめれば線形1枚と同じ**。表現力は1ミリも増えない
- 線形の限界：各入力の「持ち点」を**足す**ことしかできない。窓に3文字入れても、**3文字の組み合わせ**（「この並びのときだけ」）には反応できない

非線形の最小部品 — ReLU

- そこで、線形のあいだに**直線ではない**部品を挟む。最小の候補：負の数を0に折るだけの関数

$$\mathrm{ReLU}(x) = \max(0,\ x)$$



Code

```
import torch
import torch.nn as nn

x = torch.tensor([[-2.0, -0.5, 0.0, 1.0, 3.0]])
print(nn.ReLU()(x))          # 負だけ0に折られる
```

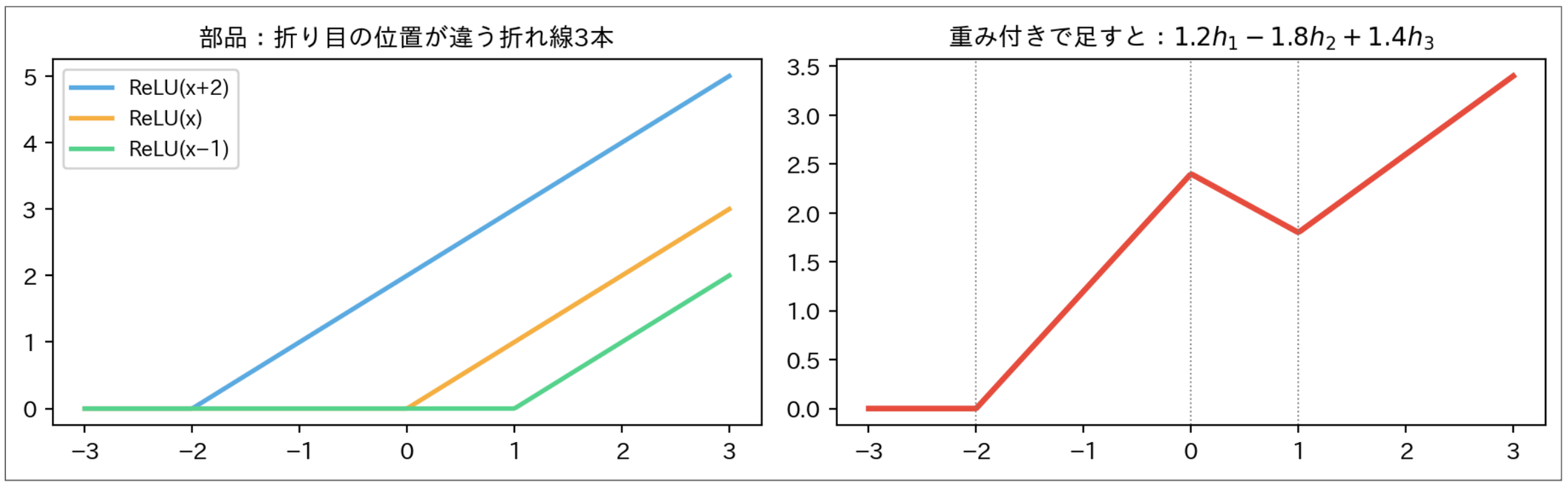
Result

```
tensor([[0., 0., 0., 1., 3.]])
```

- 名前は **ReLU**（Rectified Linear Unit）。中身はこれだけ — だが0のところに**折り目**がある

6

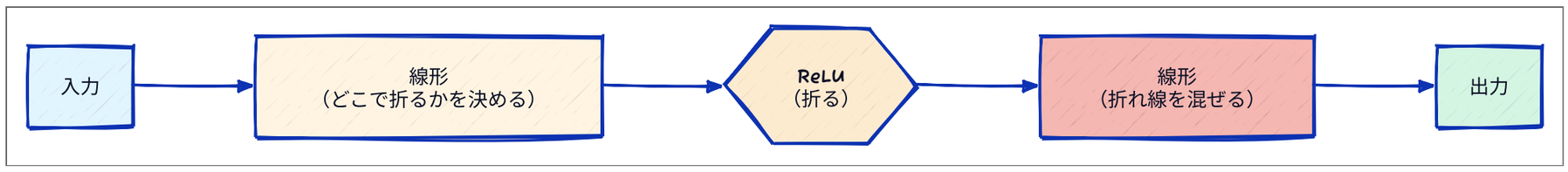
折り目を足すと、曲がった規則が作れる



- ReLU 1個＝折り目1個。**重み付きで足し合わせる**と、**折り目の数だけ曲がる折れ線**が作れる
- 折る位置（1枚目の線形）も混ぜ方（2枚目の線形）も、勾配降下で学習される — 折り紙は機械が自分で折る

7

線形 → 折る → 線形 ＝ MLP



- この3段重ねが**多層ネットワーク**（**MLP multi-layer perceptron**）
- 今日の窓モデルは 24次元 → **64**（折り目64本）→ 75クラス
- **線形1枚では書けなかった「組み合わせの規則」**（「か」の次は前後しだいで「わ」にも「み」にも）がこれで書ける

データを窓の形に切り直す

- 名前の前に窓ぶんの「・」を敷いてから、1文字ずつずらして切る

Code

```
import pandas as pd

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}

def windows(name_list, ctx):
    xs, ys = [], []
    for name in name_list:
        seq = [0] * ctx + [stoi[c] for c in name] + [0]
        for i in range(len(seq) - ctx):
            xs.append(seq[i:i + ctx])   # 窓
            ys.append(seq[i + ctx])     # その次の1文字（正解）
    return xs, ys

xs, ys = windows(["しぶや"], ctx=3)
for x, y in zip(xs, ys):
    print("".join(chars[i] for i in x), "→", chars[y])
```

Result

```
・・・ → し
・・し → ぶ
・しぶ → や
しぶや → ・
```

- 「・・・→し」は「何もない状態から『し』で始まった」という訓練例になる

訓練と評価を分けておく

- 今日は「本当に上手くなったか」を測りたい。ところが**学習に使ったデータでの成績は当てにならない** — **丸暗記でも点が取れてしまう**から
- そこで最初にデータを2つに分ける：

  → **train**（訓練用）— 学習に使う

  → **val**（評価用 **validation**）— 採点だけに使い、モデルには**見せない**
- 見せていない175件で当てられたら、それは暗記ではなく**実力**

Code

```
import pandas as pd
import torch

names = pd.read_csv("data/chimei.csv")["yomi"].tolist()

g = torch.Generator().manual_seed(42)
perm = torch.randperm(len(names), generator=g).tolist()
n = int(len(names) * 0.9)
train_names = [names[i] for i in perm[:n]]   # 1,572件で学習
val_names = [names[i] for i in perm[n:]]     #   175件で採点
print(len(train_names), len(val_names))
```

Result

```
1572 175
```

MLP言語モデルを組んで学習

Code

```
import torch.nn as nn
import torch.nn.functional as F

torch.set_num_threads(1)
ctx = 3
X, y = map(torch.tensor, windows(train_names, ctx))
Xv, yv = map(torch.tensor, windows(val_names, ctx))

torch.manual_seed(0)
model = nn.Sequential(nn.Embedding(75, 8),          # 文字→8次元ベクトル
                      nn.Flatten(1),                # 3本を1本(24次元)につなぐ
                      nn.Linear(8 * ctx, 64),
                      nn.ReLU(),
                      nn.Linear(64, 75))
opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
gb = torch.Generator().manual_seed(0)

for it in range(1, 3001):
    ix = torch.randint(len(X), (256,), generator=gb)
    opt.zero_grad()
    loss = F.cross_entropy(model(X[ix]), y[ix])
    loss.backward()
    opt.step()

with torch.no_grad():
    print(f"train loss: {F.cross_entropy(model(X), y).item():.4f}")
    print(f"val   loss: {F.cross_entropy(model(Xv), yv).item():.4f}")
```

Result

```
train loss: 1.8601
val   loss: 2.6139
```

- バイグラムの2.34を（訓練側では）大きく割った。学習ループは今日も3拍子のまま

生成して聞き比べる

- 分布からのサイコロは **`torch.multinomial(確率, 1)`** — bigram の回の **`np.random.choice(p=...)`** の torch 版

Code

```
torch.manual_seed(1)
for _ in range(10):
    window, s = [0] * ctx, ""
    while True:
        logits = model(torch.tensor([window]))
        i = torch.multinomial(F.softmax(logits, dim=-1), 1).item()
        if i == 0:
            break
        s += chars[i]
        window = window[1:] + [i]     # 窓を1文字ぶんスライド
    print(s)
```

Result

```
おおじたわらまち
つるぎし
きょうおんし
たかさらちょう
みなみちょう
しわばし
きたがたかきたなきまち
つまえちょう
おおのし
しべしまちやまたまち
```

数えるバイグラム（前々回）	MLP（今日）
きたやまちょう	おおじたわらまち
なごうし	つるぎし
とんむらそんだてしかむら	きょうおんし
さがさっさとかわし	たかさらちょう

- 「村を2回付ける」ような迷子が減り、**最後まで一貫した名前**が増えた

うますぎる名前には気をつける

Code

```
made = ["おおじたわらまち", "つるぎし", "きょうおんし", "みなみちょう", "おおのし"]
for s in made:
    print(s, "→", "実在する" if s in names else "新作")
```

Result

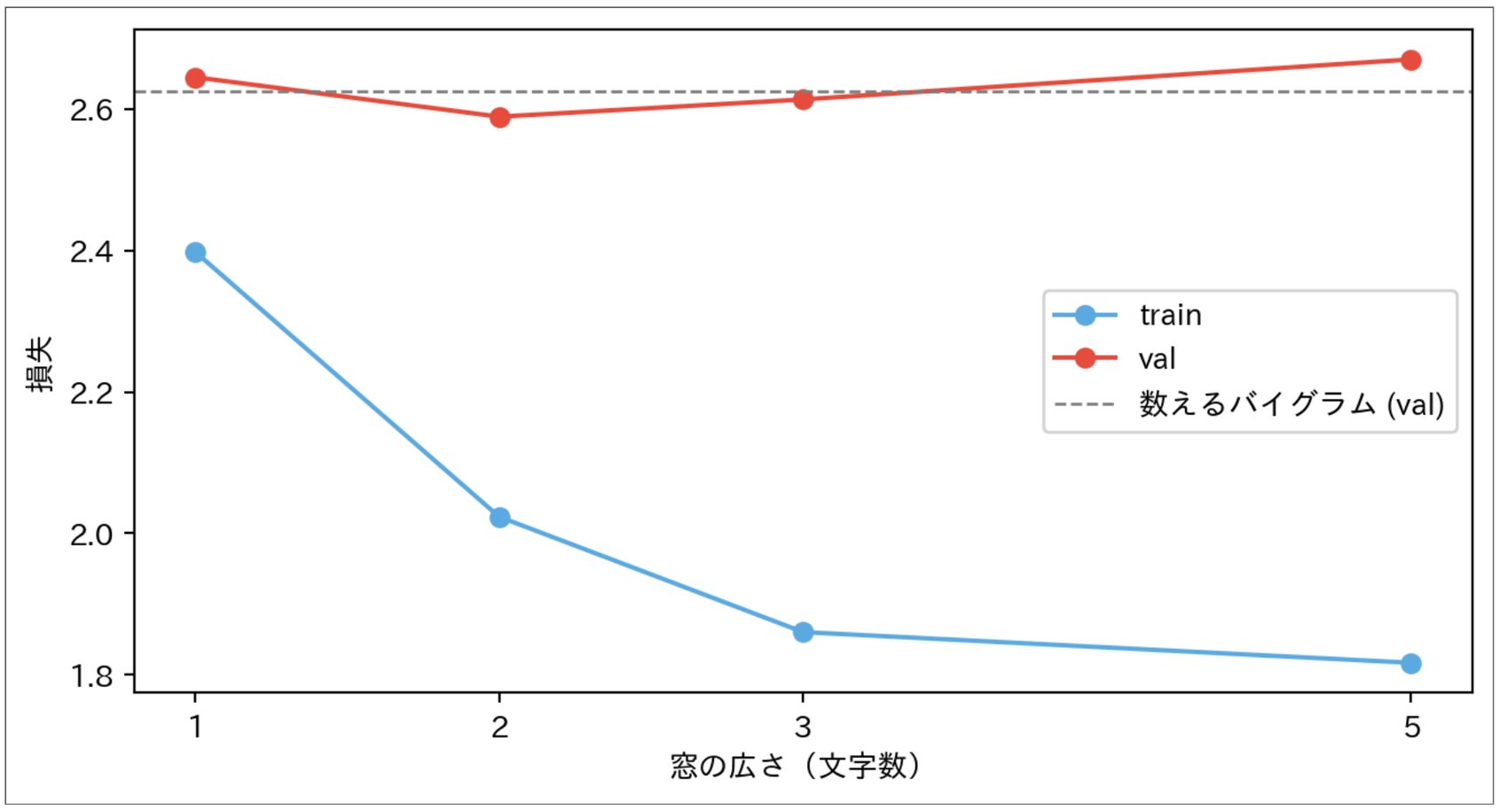
```
おおじたわらまち → 新作
つるぎし → 新作
きょうおんし → 新作
みなみちょう → 実在する
おおのし → 実在する
```

- 10件中2件は**実在の名前をそのまま再現**していた（みなみちょう・おおのし）
- 「上手くなった」と「覚えただけ」の区別 — どこかで聞いた話が始まる

クイズ：窓を広げれば広げるほど良い？

クイズ

窓を 1 → 2 → 3 → 5 文字と広げると、**見せていないデータでの損失**はどうなる？ 予想してから、めくる。



- train は下がり続けるのに、**val は窓2で底を打って悪化に転じる**（前コースで多項式の次数を上げすぎたときと同じ形）

過学習 — 暗記で点を稼ぎ始めた

- 訓練データにだけ細かく合わせすぎて、**初見のデータでかえって下手になる**現象を**過学習**（**overfitting**）と呼ぶ
- 窓を広げる＝「この並びのときはこの文字」という**丸暗記**がしやすくなる。名前はたった1,572件 — 暗記で train の点は稼げるが、新しい名前には通用しない
- グラフの読み方：**train と val の線の開きが、暗記の進み具合**
- 対策の発想：データを増やす／モデルを控えめに／**val を見て早めに止める**

ノート

窓の組数は窓幅によらず**10,288組のまま**（1名前あたり「文字数+1」個）。 データは増えていないのに表現力だけ上がる — 過学習の典型的な条件。

それでも「窓」には根本的な限界がある

- 窓の外は**永遠に見えない**。窓5でも、6文字前はもう存在しない
- 窓を広げると入力の次元も暗記のリスクも増える — 力ずくに未来はない
- 地名（平均5.5文字）ならこれでいい。だが最終目標は**文章**：

> 「吾輩は猫である。名前は**まだ**……」

- 「まだ無い」と続けるには、9文字前の「名前」を見る必要がある
- **全部見えて、そのとき必要なところだけ効かせる**仕組みが要る — それが次章の主役

ⓘ ここまでで幹1が完結

数える→学習する→埋め込む→窓を広げる。「架空の地名メーカー」はここで一区切り。 ここから先は、この部品たちを持って**文章**の世界（『吾輩は猫である』）へ移る。

Tweak 窓と部品で遊ぶ



- **学習のコード** を書き換えて、生成まで続けて実行

1	`ctx = 3` → `ctx = 1`	バイグラム相当に退化。生成名の迷子っぷりは戻る？
2	`nn.Linear(8 * ctx, 64)` の **64 → 8**	表現力を絞ると train/val はそれぞれどっちに動く？
3	`torch.manual_seed(1)`（生成側）→ **別の数**	別の名前セット。お気に入りの新作地名を探そう

ここで観察

- `ctx = 1` に戻すと val 損失は約2.65 — 数えるバイグラム（2.62）とほぼ同じ所に落ちる。**見える文脈が同じなら、たどり着く実力も同じ**
- 生成コードは窓幅を変えてもそのまま動く（`[0] * ctx` と「窓スライド」で書いてあるから）

ベスト新作コンテスト



シードを変えながら生成し、**いちばん実在しそうな新作地名**を1つ選んで提出せよ。

0. 生成側の **`torch.manual_seed`** を変えながら量産
1. 候補が実在しないことを確認（**`"○○" in names`** が **`False`**）
2. 「名前」と「推しポイントひとこと」をセットで提出
3. 全員ぶんを並べて投票 — 一番「実在しそう」な名前が優勝

> **ⓘ ノート**
>
> 最終回には同じ形式で**文豪テスト大会**（偽物の文章で騙し合う）をやる — 今日はその練習試合。

まとめ

- 窓＝直前 n 文字を埋め込み→連結→MLP。部品は埋め込み・線形・ReLU の3つだけ
- 生成の質は明確に向上（迷子が減る）。ただし val の改善はわずかで、**窓を広げすぎると過学習**
- 実在名の「丸暗記再現」も出始めた — 上達と暗記は別物
- 窓には根本的限界：**窓の外は見えない**

現在の設計図　直前 n 文字 → 埋め込み → MLP → 語彙スコア

> **ⓘ 次回予告**
>
> 文章の中で「どこを見るべきか」は、そのときどきで違う。 固定の窓ではなく、**モデル自身に「見る場所」を選ばせる** — Attention の登場。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：自分の入力の「次」を当てさせる

学習済みの MLP（ctx=3）に好きな3文字（ひらがな）を入れ、次の1文字の**予想トップ3**を確率つきで表示せよ。

0. 本編のコードで学習まで実行（**chimei.csv** を **`data/`** に配置）
1. 3文字を **`stoi`** で番号にして **`model(torch.tensor([番号列]))`** でスコアを出す
2. **`F.softmax`** で確率にし、**`topk(3)`** で上位3件を「文字：確率」で表示（**`p.topk(3)`** は「上位3つの値と番号」の組を返す）
3. 「かわさ」「よこは」で試し、直感と合うか確かめる

問題2：窓と組数

窓幅 1・3・5 のそれぞれで訓練データの組数（**`len(X)`**）を表示し、**同じ数になる理由**を1行で説明せよ。

解答例

▸ 解答例を見る

Result

```
かわさ → [('と', 0.391), ('き', 0.339), ('か', 0.097)]
よこは → [('ま', 0.763), ('ち', 0.107), ('し', 0.044)]
窓1: 11428組
窓3: 11428組
窓5: 11428組
```